

Arquiteturas Alternativas

QUESTÃO 1

ENUNCIADO:

Por que arquiteturas alternativas, mesmo desempenhando melhor em certos cenários, não conseguem substituir o transformer padrão?

RESPOSTA:

Apesar de arquiteturas alternativas como RetNet, RWKV, Mamba e variações como o Differential Transformer apresentarem vantagens claras em cenários específicos, como uma maior eficiência em sequências longas, custo de inferência mais baixo ou melhor estabilidade numérica, elas ainda não conseguem substituir o Transformer padrão porque ele oferece um equilíbrio robusto entre desempenho, generalidade e maturidade. O Transformer já foi extensivamente testado e otimizado em uma enorme variedade de domínios (NLP, visão, multimodalidade), e ao longo dos anos consolidou um conjunto de ferramentas e técnicas que vão muito além da arquitetura em si. Exemplos para isso são kernels otimizados como FlashAttention, bibliotecas padrão em diferentes frameworks, definições consolidadas de pré-treino, técnicas de escalonamento de contexto, quantização, paralelismo e servimento em produção. Migrar para uma arquitetura alternativa exigiria refazer grande parte dessa infraestrutura, desenvolvendo novos kernels, novas rotinas de treinamento e novas otimizações. Modelos como o Mamba, por exemplo, dependem de implementações específicas de SSM seletivo e algoritmos paralelos customizados para explorar totalmente seu potencial em GPU, o que ainda não está tão maduro quanto o Transformer. Além disso, muitas dessas arquiteturas são desenhadas para atacar gargalos bem definidos, como custo quadrático da atenção ou uso de memória em contexto longo, e, por isso, acabam não superando o Transformer em todas as tarefas relevantes. Como o Transformer já demonstrou desempenho competitivo e estável em ampla escala, adotar uma arquitetura mais recente pode introduzir um risco, o que desincentiva sua substituição completa e favorece um cenário de uso complementar ou híbrido em vez de um abandono da arquitetura atual.

QUESTÃO 2

ENUNCIADO:

Compare o uso de técnicas posteriores ao treino para mitigar problemas gerados por transformers com o uso de arquiteturas alternativas.

RESPOSTA:

Arquiteturas alternativas exigem refazer justamente a parte mais cara de todo o pipeline, que é o pré-treino massivo do modelo em escala de bilhões de tokens, enquanto técnicas posteriores ao treino conseguem reaproveitar o investimento já feito nos Transformers. Treinar uma arquitetura nova grande o suficiente para competir seriamente com os modelos dominantes implica gastar de novo uma fortuna em compute, engenharia e tempo, sem garantia de que o resultado final será claramente superior. Além disso, praticamente toda a infraestrutura de software e hardware atual, como kernels otimizados e bibliotecas, foram feitas com base no Transformer, o que aumenta ainda mais o custo de migrar. Em contrapartida, estratégias de pós-treino, como instruction tuning, RLHF/RLAIF, finetuning supervisionado, filtros e guardrails, ou ainda soluções de sistema como RAG, atuam “em cima” de um backbone já treinado, com conjuntos de dados menores, menos passos de otimização e muito menos risco, pois se algo não funcionar tão bem, você ajusta ou descarta aquele método, não o modelo inteiro. Por isso, mesmo sabendo das limitações dos Transformers, a tendência é priorizar mitigações posteriores ao treino, que são incrementalmente baratas e compatíveis com a infraestrutura existente, em vez de apostar repetidamente em arquiteturas completamente novas.

QUESTÃO 3

ENUNCIADO:

Como o mecanismo de atenção no Transformer se diferencia do modo como as redes recorrentes processam e utilizam informação ao longo da sequência?

RESPOSTA:

Transformer e redes recorrentes lidam com dependências em sequência de maneiras fundamentalmente diferentes. Nas RNNs, como LSTMs ou GRUs, a informação flui passo a passo, de forma que cada novo token é processado usando o estado oculto anterior, que funciona como uma memória comprimida do passado. Isso significa que a dependência é sequencial, e o modelo só acessa informações antigas indiretamente, através de resumos sucessivos. Esse processo pode ser limitado por problemas como vanishing e exploding gradients, dificultando a captura de relações de longo alcance, além de tornar o processamento pouco paralelizável. No caso do Transformer, o mecanismo de atenção permite que cada posição da sequência “olhe diretamente” para qualquer outra posição, sem precisar passar por um estado intermediário. Na prática, isso significa que o modelo calcula pesos de

relevância entre todos os pares de tokens, permitindo capturar dependências longas de forma mais explícita e eficiente. Além disso, como todas as posições são processadas simultaneamente, Transformers se beneficiam muito de paralelização em GPU, alcançando desempenho superior em tarefas de linguagem. Em resumo, enquanto RNNs dependem de um fluxo sequencial e memória comprimida, Transformers usam atenção para acessar diretamente todo o contexto, ampliando capacidade de representação e eficiência computacional.

QUESTÃO 4

ENUNCIADO:

Explique por que em arquiteturas alternativas há interesse em atenção $O(T)$ em vez de $O(T^2)$.

RESPOSTA:

Há um interesse crescente em arquiteturas lineares no número de tokens de entrada (em vez da self-attention $O(T^2)$) porque muitos cenários atuais estouram o orçamento de tempo e memória quando o comprimento/área de entrada cresce: contexto longo em linguagem, imagens de alta resolução em visão, áudio/séries temporais contínuos. Em todas essas frentes, formar um mapa de atenção $T \times T$ é o gargalo. Modelos com custo $O(T)$, como RWKV, Mamba e RetNet, substituem a interação par-a-par por estados/varreduras recursivas ou kernels estruturados, preservando dependências de longo alcance sem materializar a matriz quadrática. Do ponto de vista prático, isso viabiliza escalar T mantendo FLOPs/memória proporcionais, deploy em dispositivos com RAM/VRAM limitada e streaming com latência e estado por passo constantes. No treino, a redução de memória pode liberar batch maior ou modelos mais largos no mesmo orçamento. Porém, existem trade-offs, pois essas camadas costumam exigir scans sequenciais (menos paralelismo que grandes matmuls da atenção), e o ecossistema é mais maduro para Transformers. Em tarefas com T moderado e demanda por alto throughput, Transformers ainda são convenientes. Mas quando T domina o custo (long-context, 2K–4K+ em visão, fluxos contínuos), as alternativas lineares oferecem uma rota de escala sustentável, mantendo globalidade e reduzindo drasticamente o uso de memória.

QUESTÃO 5

sENUNCIADO:

Em arquiteturas alternativas, quais são as vantagens de reparametrizar um parâmetro (e.g. definir uma equação para ele) em vez de tratá-lo simplesmente como um escalar aprendível?

RESPOSTA:

Em arquiteturas alternativas, reparametrizar um parâmetro, ou seja, defini-lo como função de outros termos, como tempo, posição, entrada ou até de outros parâmetros, em vez de tratá-lo apenas como um escalar aprendível independente, traz vantagens conceituais e práticas. Em primeiro lugar, isso força o modelo a respeitar estruturas que já se sabe que são desejáveis, como decaimento no tempo, normalização, limites de estabilidade, simetria e positividade. Com isso, o espaço de soluções que o otimizador pode explorar fica mais restrito a comportamentos plausíveis, o que tende a melhorar a capacidade de generalização e a estabilidade numérica, já que o parâmetro deixa de assumir valores arbitrários e passa a seguir uma forma controlada, como um decaimento exponencial suave ou uma função que naturalmente satura e evita explosões. Em segundo lugar, a reparametrização costuma tornar o processo de otimização mais bem comportado, pois substitui variáveis diretamente instáveis, como ganhos multiplicativos grandes, por representações em um espaço onde o gradiente se propaga de maneira mais estável, como trabalhar com o log de um ganho ou com uma variável angular em vez do valor bruto. Em terceiro lugar, reparametrizar permite compartilhar estrutura entre diferentes partes do modelo, por exemplo tempos de esquecimento similares entre camadas ou dependências suaves de posição ao longo da rede, o que reduz o número total de parâmetros e favorece o aprendizado em cenários com menos dados. Em arquiteturas como Mamba, RetNet ou mecanismos de atenção modificados, reparametrizar taxas de decaimento, coeficientes de mistura e termos de normalização é uma estratégia central para garantir estabilidade, eficiência em contexto longo e um comportamento mais interpretável, evitando deixar esses fatores como escalares soltos e arbitrários.